哈爾濱工程大學

第 16 讲 / 全 17 讲

C 语言文件操作详解

让程序拥有记忆：把数据写到硬盘上，下次开机还在

FILE 指针

文件在程序里的遥控器，一切操作都由它指挥

打开模式

写、读、追加三种进门方式，选错就会出事

格式化读写

fprintf 与 fscanf ，比老朋友只多一个 fp

结构体落盘

学号、姓名、成绩整整齐齐存进文件

主讲教师：郎大鹏

本节目标

哈尔滨工程大学

1. 说清为什么要用文件：内存里的数据随程序结束而消失
2. 区分文本文件与二进制文件，知道本课用文本方式
3. 掌握 FILE 指针与 fopen 、 fclose 的正确配对
4. 会用 fprintf 、 fscanf 按格式读写，并能检查打开是否失败
5. 用 while 循环配合返回值 EOF 把整个文件读完
6. 把结构体数组存进文件再读回来，做出成绩管理系统的雏形

01

文件入门

为什么要文件 · 文本与二进制 · FILE 指针 · fopen 模式 · fclose

本部分关键词： FILE 指针是文件的遥控器， fopen 开机， fclose 关机保存。

学完这六页，你能让数据真正留在硬盘上，不再随程序结束而消失。

为什么需要文件

痛点 前面所有程序，数据都住在内存里。程序一关，房间被回收，数据跟着消失。

```
// 只有变量的程序：记不住
int score = 0;
scanf("%d", &score);
printf("这次是 %d", score);
// 屏幕上看到 95
return 0;
// 程序结束，score 这间房被回收
// 再运行一次，上一次的 95 再去哪找？
// 答案：找不回来，除非把它写进文件
```

内存：临时工

变量住在内存条上，速度快，但一断电、一结束就清空。

硬盘文件：铁饭碗

数据写在磁盘的文件里，关机、重启、明年打开都还在。

所以要有文件

想让程序「记得你」，就得把数据从内存搬到文件里。

文本文件 vs 二进制文件

本课用文本文件

哈尔滨工程大学

硬盘上的文件分两类。判断方法极简单：用记事本打开，能看懂的是文本文件，一堆乱码的是二进制文件。

文本文件：看得懂

里面存的全是字符，本质是字符串。

记事本打开直接能读、能改。

读写函数： fprintf 、 fscanf 、 fputs 、 fgets 。

二进制文件：看不懂

原样搬运内存里的字节，不是给人看的。

记事本打开是一坨乱码，只有程序认得。

读写函数： fread 、 fwrite （本课不展开）。

```
//  文本文件里真实存的内容，打开就是这样
张三  95  18
李四  88  19
//  用什么格式写进去，就得用什么格式读出来：配套使用  %s  %d  %d
```

FILE 指针：文件的遥控器

呼应指针课

哈尔滨工程大学

```
// FILE 类型由 stdio.h 提供，直接用
FILE *fp;
// fp 拿到的是地址，指向一块文件信息区
fp = fopen("data.txt", "w");
// 之后所有动作都靠 fp 这个遥控器中转
fprintf(fp, "%d", 100);
// 写完之后把遥控器还回去
fclose(fp);
// 手错一点点都不会指向文件，所以类型必须写 FILE *
```

FILE 是个新类型

跟 int、double 一样是类型，只是它装的是「描述一个文件的一堆信息」。

为什么写成星号

FILE 那块信息很大，不能搬来搬去。用指针记住它在哪，正如遥控器和电视。

一句记住它

fp 不是文件内容，是通往文件的那扇门。所有读写函数第一个参数都是它。

fopen 与打开模式

开门要有目的：是写、是读、还是接着往后写。目的写在第二个参数里，用字符串给出。

```c
FILE *fp;
fp = fopen("data.txt", "w");
// w：文件没有就新建，有就清空重写
fprintf(fp, "%d", 88);
fclose(fp);
fp = fopen("data.txt", "r");
// r：文件必须已经存在，否则失败
fp = fopen("data.txt", "a");
// a：没有就新建，有就写到末尾
//  模式写错或文件不存在， fp  都会是  NULL
```

"w" 写 write
没有就建，有就清空。覆盖危险。

"r" 读 read
文件必须已存在，否则打开失败。

"a" 追加 append
原内容保留，新数据写到末尾。

fclose 不能忘

高频丢分点

```
FILE *fp;
fp = fopen("log.txt", "w");
fprintf(fp, "%d%c", 100, 10);
// 字符 10 就是换行键，写进文件正合适
fprintf(fp, "%d%c", 200, 10);
// 此刻数据可能还攒在缓冲区里
fclose(fp);
// 这一句之后，硬盘上才真的有 100 和 200
// 用完不还，别的程序可能就打不开这个文件
```

缓冲区：攒够再送

写数据不会立刻落盘，先攒在内存的缓冲区里，攒够一批才一起搬。

fclose 干两件事

把缓冲区里剩下的数据一次性写完，再把遥控器还给系统。

忘了会怎样

文件可能是空的、少一半内容，或者被占着不让别人打开。

打开失败怎么办

if + 指针综合

哈爾濱工程大學

```c
FILE *fp;
int n = 0;
fp = fopen("score.txt", "r");
// 开门失败时 fopen 返回 NULL（空指针）
if (fp == NULL) {
    printf("打开失败，程序结束");
    return 1;
}
// 走到这里说明 fp 有效，可以放心读写
fscanf(fp, "%d", &n);
fclose(fp);
```

什么时候会失败

文件根本不存在、路径写错、被别的程序占用、没有权限。

失败时的签名

fopen 返回一个特殊地址 NULL，意思是「哪都没指着」。

标准套路

先 if (fp == NULL) 拦一道，确认有效再往下走。

02

格式化读写

fprintf · fscanf · 游标 · EOF · 按行读写

本部分关键词： printf 加一个 fp 就是 fprintf ， scanf 加一个 fp 就是 fscanf 。

再搞懂「读到哪就从哪继续」的游标，就能把整个文件读完。

fprintf ：往文件里 printf

只多一个参数

```
FILE *fp;
int a = 95;
double h = 1.75;
fp = fopen("out.txt", "w");
// 比 printf 只多一个 fp，写在最前面
fprintf(fp, "%d", a);
fprintf(fp, "%.1f", h);
fprintf(fp, "%d%c", a, 10);
// 数字 10 打印成 %c，文件里就是换行
fclose(fp);
```

对照 printf

printf 默认往屏幕送；fprintf 在前面指明「往哪个文件送」。

格式串完全照搬

%d 、%f 、%c 、%s 这些老朋友一个不少，用法一模一样。

写多个数据

一次写多个也行，数据之间要有空格隔开，不然读回来会粘成一坨。

fscanf ：从文件里 scanf

```
FILE *fp;
int a = 0;
double h = 0;
char name[20];
fp = fopen("out.txt", "r");
fscanf(fp, "%d", &a);
fscanf(fp, "%lf", &h);
//  数组名本身就是地址，这一项不用加符号
fscanf(fp, "%s", name);
printf(" 读到  %d", a);
fclose(fp);
```

对照 scanf

scanf 从键盘读， fscanf 从 fp 指的那个文件读，其余完全一样。

变量前的符号照旧

普通变量照写地址符，字符数组直接给数组名，规则跟 scanf 一致。

格式要能对上

写的时候用 %s %d ，读的时候也得用同样的顺序和间隔，错位就读乱。

文件的游标：读到哪从哪继续

每个被打开的文件内部藏着一个游标，它记住「下一次该读哪里」，读一次就往后挪一格。

第一步	第二步	第三步	第四步	再读一步
85	90	78	61	到末尾

```
fp = fopen("data.txt", "r");
fscanf(fp, "%d", &x);
// 第一次读到 85，游标停在 90 前面
fscanf(fp, "%d", &x);
// 第二次读到 90，游标又往后挪一格
fclose(fp);
```

不用你操心位置

系统自动帮你挪，反复调用同一个 fscanf 就行。

重头再来要重开

游标到底还想再读，只能 fclose 后重新 fopen。

fscanf 的返回值与 EOF

回应 scanf 课的伏笔

哈尔滨工程大学

```c
FILE *fp;
int x = 0, ret = 0;
fp = fopen("data.txt", "r");
// fscanf 是有返回值的，别丢掉
ret = fscanf(fp, "%d", &x);
printf("返回值 %d", ret);
// 读到了就返回成功读入的数据个数：1
ret = fscanf(fp, "%d", &x);
// 已经没得读，返回 EOF，也就是 -1
printf("返回值 %d", ret);
fclose(fp);
```

返回值是什么意思

返回成功读到的数据个数。用一次 %d 读一个整数，成功就返回 1。

EOF 就是 -1

End Of File 的缩写，读到文件末尾时返回它。本质是 -1，所以能直接比。

当年的伏笔兑现

scanf 课说过的返回值在这里派上用场：它是判断是否读完的唯一可靠信号。

while 循环把整个文件读完

本节最重要的一页

哈尔滨工程大学

```c
FILE *fp;
int x = 0, sum = 0, cnt = 0;
fp = fopen("nums.txt", "r");
// 读一次，判断一次，不是 EOF 就继续
while (fscanf(fp, "%d", &x) != EOF) {
    sum = sum + x;
    cnt = cnt + 1;
}
printf("共 %d 个", cnt);
printf("总和 %d", sum);
fclose(fp);
```

不用知道文件多大

文件里有 3 个数还是 3000 个数，这一套写法通吃。

循环条件在干活

把 fscanf 写进条件里，顺手读了数据，又顺手判断了是否读完。

累加器老搭档

循环中做求和、计数、打擂台，跟以前讲过的套路完全一致。

fgets 读一行 / fputs 写一行

```
FILE *fp;
char line[100];
fp = fopen("note.txt", "w");
// fputs：把一整串字符交给文件
fputs("你好哈工程", fp);
fclose(fp);
fp = fopen("note.txt", "r");
// fgets：第二个参数是数组容量，别超了
fgets(line, 100, fp);
printf("%s", line);
fclose(fp);
```

整行处理更顺手

一句话里带着空格时，%s 会在空格处中断，整行读写才不会。

fgets 的三参数

数组名、最大能装多少个字符、从哪个文件读。顺序别记反。

读不到返回 NULL

也可以写成 while (fgets(...) != NULL) 一行一行读到底。

03

文件与结构体综合

结构体落盘 · 读回来 · feof · 追加日志 · 成绩管理系统

本部分关键词：结构体和文件是天生一对，一个负责组织数据，一个负责长期保存。

学完这六页，你能写出真正像样的学生成绩管理系统。

把结构体写进文件

逐成员写

```
// 假设已经录好三个人的信息
struct Student s[3];
FILE *fp;
int i;
fp = fopen("stu.txt", "w");
for (i = 0; i < 3; i++) {
    fprintf(fp, "%s %d%c", s[i].name,
                  s[i].score, 10);
// 每行一名学生：张三 95
}
fclose(fp);
```

逐成员往外写

结构体不会自己变身文本，要把每个成员挨个 fprintf 一遍。

一个学生占一行

末尾用 %c 输出 10 换行，读的时候才分得清谁是谁。

中间要加空格

各成员之间留空格或分隔符，粘成一团就读不回去了。

从文件读回结构体数组

```
struct Student s[100];
FILE *fp;
int i = 0, n = 0;
fp = fopen("stu.txt", "r");
while (fscanf(fp, "%s %d", s[n].name,
              &s[n].score) != EOF) {
    n = n + 1;
//  每成功读一行，记录条数加一
}
printf("读回 %d 条", n);
fclose(fp);
```

n 既是下标又是计数器

读到第几条就往 s[n] 里放，读完之后 n 正好是总人数。

格式要和写时一致

写的时候是「名字 成绩」，读的时候就必须是同样的 %s %d 。

数组要留够

s[100] 是上限，实际存了几个人由 n 说了算，遍历时只走到 n 。

feof 判断文件结束

容易用错

```
FILE *fp;
int x = 0, sum = 0;
fp = fopen("nums.txt", "r");
while (1) {
    fscanf(fp, "%d", &x);
    if (feof(fp)) {
        break;
    }
    sum = sum + x;
}
fclose(fp);
```

feof 是个标记

文件读到头时会被打上一个「到末尾了」的标记，feof 专门查这个标记。

必须在读之后用

标记是「读失败之后」才打上的，先读再问，别指望它能预测未来。

推荐还是用返回值

判 返回值 是否等于 EOF 更省心，feof 作为补充认识就行。

追加模式记录日志

"a" 模式

```
FILE *fp;
char name[20];
int score;
fp = fopen("log.txt", "a");
// a：老记录保留，新内容排在末尾
if (fp == NULL) {
    printf("打开失败");
    return 1;
}
scanf("%s %d", name, &score);
fprintf(fp, "%s %d%c", name, score, 10);
fclose(fp);
```

越跑记录越多

每运行一次加一条，关机也不丢。这就是日志的思想。

换成 "w" 就惨了

写模式每次都会清空文件，最后只剩最后一条记录。

文件不存在也行

追加模式下文件不在就新建一个，跟写模式一样，不必提前准备。

成绩管理系统雏形

前面所有知识的合体

哈尔滨工程大学

一个完整的套路：先把数据录进来，写进文件；下次运行时从文件读回来，再做统计。

1 录入

循环 scanf 填结构体数组

2 存文件

用 "w" 逐条 fprintf 写盘

3 读回来

用 "r" 配合 EOF 读成数组

4 做统计

遍历求平均、找最高分

```
// main 里把四步串起来，每个动作是一个函数
int n = 0;
struct Student stu[100];
n = input(stu);          // 1 键盘录入，返回人数
save(stu, n);            // 2 全部写进 stu.txt
n = load(stu);           // 3 从文件读回，重新得到人数
report(stu, n);          // 4 遍历统计并输出
```

文件操作四步总结

背下来就不慌

哈尔滨工程大学

1 打开

决定是读还是写，拿到 FILE 指针。

```
fp = fopen("a.txt", "w"
```

2 检查

先看 fp 是不是 NULL，确认进门了。

```
if (fp == NULL) {
    return 1;
}
```

3 读写

格式化读写，配 while 判断返回值。

```
fprintf(fp, "%d", x);
fscanf(fp, "%d", &x);
```

4 关闭

用完立刻 fclose，数据才真正落盘。

```
fclose(fp);
```

顺序不能乱，缺一不可

没检查就读写，程序会在错误的文件指针上崩溃；

没 fclose 就收工，数据可能压根没写进去。

养成四步走的习惯，文件操作就不会出岔子。

例题精讲

例 1 把 1 到 100 这一百个整数写进文件 num.txt ，每行一个。

```c
int main(void) {
    FILE *fp;
    int i;
    fp = fopen("num.txt", "w");
    // 循环一百次，每次写一个数
    for (i = 1; i <= 100; i++) {
        fprintf(fp, "%d%c", i, 10);
    }
    fclose(fp);
    printf("写完了");
}
```

运行结果

屏幕：写完了

num.txt 里的内容：

```
1
2
3
...
100
```

要点：%c 输出字符 10 就是换行。打开模式用 "w" ，跑几次都不会重复。

例题精讲

例 2 把 num.txt 里的数全部读出来，求和并算平均分。

```c
int main(void) {
    FILE *fp;
    int x = 0, sum = 0, cnt = 0;
    fp = fopen("num.txt", "r");
    // 一直读到文件末尾为止
    while (fscanf(fp, "%d", &x) != EOF) {
        sum = sum + x;
        cnt = cnt + 1;
    }
    fclose(fp);
    printf("总和 %d 个数 %d", sum, cnt);
    printf("平均 %.1f", 1.0 * sum / cnt);
}
```

运行结果

```
总和 5050 个数 100
平均 50.5
```

为什么这么写：

循环条件顺手完成「读一个」和「判断是否读完」两件事。

cnt 同时是计数器，也是算平均的分母。

乘 1.0 是为了避免整数除法把小数砍掉。

例题精讲

哈爾濱工程大學

例 3 把三名学生的姓名和成绩写进文件 stu.txt ，一人一行。

```c
// struct Student { char name[20]; int score; };
int main(void) {
    FILE *fp;
    int i;
    struct Student s[3];
    // 假设三个人的信息已经录入完毕
    fp = fopen("stu.txt", "w");
    for (i = 0; i < 3; i++) {
        fprintf(fp, "%s %d%c", s[i].name,
                        s[i].score, 10);
    }
    fclose(fp);
}
```

运行结果

stu.txt 的内容：

```
张三 95
李四 88
王五 76
```

写法要点：

结构体不能整体塞进去，要一个成员一个成员地写。

末尾加 %c 和 10 ，保证一人占一行。

例题精讲

例 4 从 stu.txt 读出全部学生，找出成绩最高的那位并打印姓名。

```
int main(void) {
    FILE *fp = fopen("stu.txt", "r");
    struct Student x, top;
    top.score = -1;   //  擂主初值
    while (fscanf(fp, "%s %d", x.name,
                  &x.score) != EOF) {
        if (x.score > top.score) {
            top = x;
        }
    }
    fclose(fp);
    printf(" 最高分  %s", top.name);
}
```

运行结果

最高分 张三

思路点拨：

打擂台不必等全部读完，读一个比一个，读完整场比赛也就结束了。

结构体可以直接整体赋值： top = x ，两个成员一起搬走。

擂主初值要给成 -1 这种肯定比不过的值。

例题精讲

例 5 简易通讯录：录入一位联系人追加进 book.txt ，再把全部联系人打印出来。

```
void add_one(void) {
    FILE *fp = fopen("book.txt", "a");
    struct Contact c;
    scanf("%s %s", c.name, c.tel);
    fprintf(fp, "%s %s%c", c.name, c.tel, 10);
    fclose(fp);
}
void show_all(void) {
    FILE *fp = fopen("book.txt", "r");
    struct Contact c;
    while (fscanf(fp, "%s %s", c.name, c.tel) != EOF) {
        printf("%s %s", c.name, c.tel);
    }
    fclose(fp);
}
```

运行效果

book.txt ：

```
张三 1301111
李四 1392222
王五 1883333
```

struct Contact 含两个成员：name 和 tel 。

追加模式让每次运行都往上加一行。

EOF 控制循环，读完自动停。

常见错误盘点

哈尔滨工程大学

- ✗ 打开失败不检查就用： fp 还是 NULL 就往里写，程序直接崩。
- ✗ 写完忘记 fclose ：数据还攒在缓冲区里，打开发现是空的。
- ✗ 想追加却用写模式：旧内容被清空，文件里只剩最后一次的数据。
- ✗ fprintf 少写 fp ：写成 printf 之后，数据全跑到屏幕上去了。
- ✗ 游标已经在末尾还继续读：什么都读不到，只能 fclose 后重新 fopen 。
- ✗ 文件路径写错或漏写后缀：文件找不到， fopen 默默返回 NULL 。

小结

文件让程序第一次拥有了记忆

FILE 指针是遥控器，四步走是规矩：打开 → 检查 → 读写 → 关闭。

fprintf 、 fscanf 只比 printf 、 scanf 多一个 fp ；返回值 EOF 告诉你读完了。

结构体逐成员写进去、逐成员读回来，就能做出成绩管理系统。

下一讲：预处理与多文件工程

程序越来越大怎么办？学 #define 、头文件分工，把一个大程序拆成多个文件管理。